CprE 487/587 Lab 1 Report (Group 06)

Oct 7, 2026 · @jongwookim · Zach Dixon

1. Overview

We completed all the required parts of Lab 1 using the provided TinyImageNet CNN (`CNN_TinyImageNet.h5`, 770,216 parameters). The provided model gets 24.30% Top-1 accuracy on the validation set. When we trained the same model from scratch we got about the same accuracy (24–25%).

Part	Machine	Device
Dataset, inference, model exploration, profiling (3.3–3.6)	`linuxvdi-f26-19`	CPU (`ENABLE_GPU = False`)
Training (3.7)	GPU VM `cpre587-f26-06`	NVIDIA A40-8Q (`ENABLE_GPU = True`)

Software: Python 3.9.25, TensorFlow 2.16.2, TensorBoard 2.16.2 with `tensorboard_plugin_profile` 2.16.0.

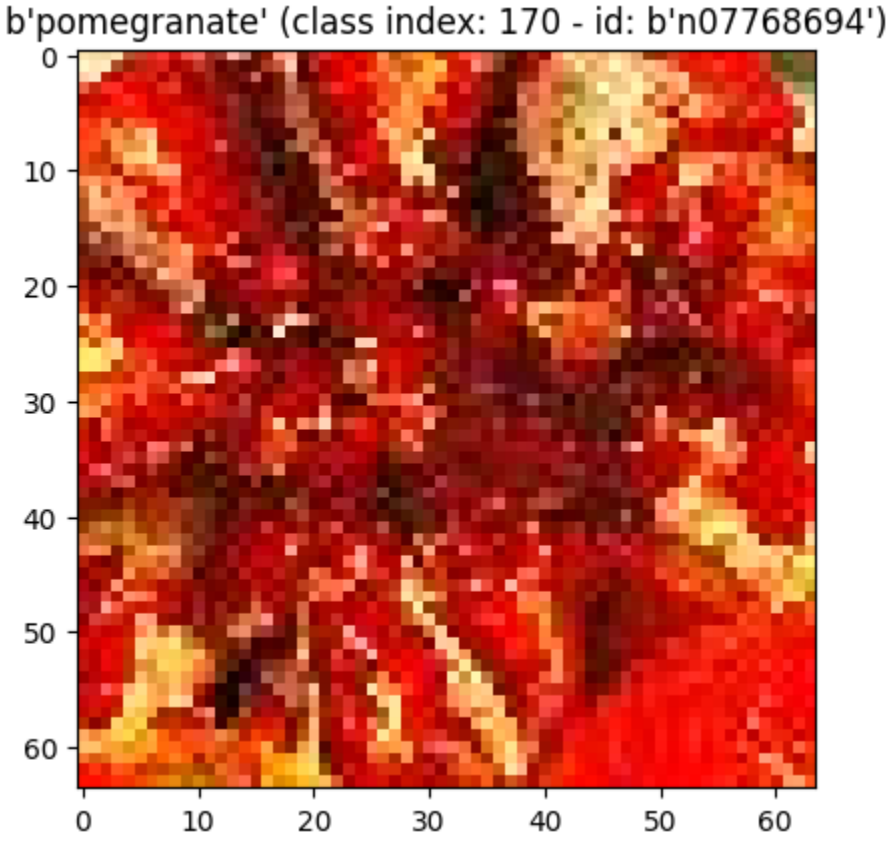
We ran the notebook in two separate kernel sessions (CPU first, then the GPU VM for training). Because of this, the first cell shows the GPU session and the execution counts of the inference cells and the training cells do not form one sequence. All the timing numbers in Section 5 were measured on the CPU.

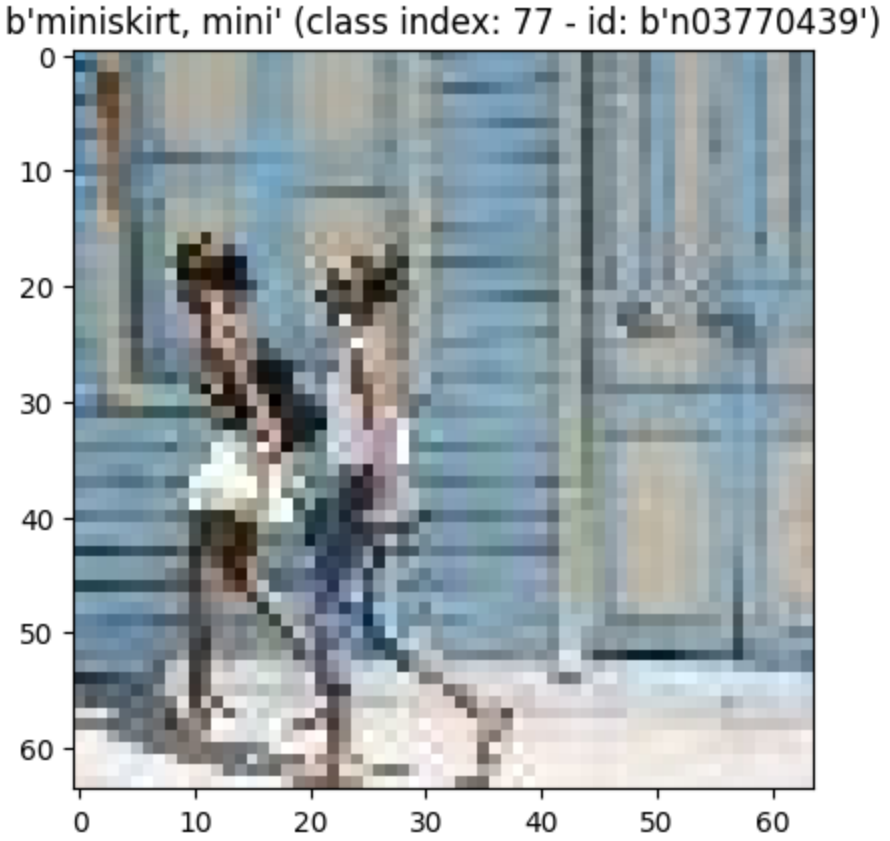
2. Dataset inspection (3.3)

Each TinyImageNet input is a 64 × 64 RGB image stored as `uint8`, which takes 12,288 bytes (64 × 64 × 3 × 1 byte). The dataset has 200 classes.

The three test images are the first three images of the unshuffled validation split, so the same images are selected on every run, including after a kernel restart.

Image	Class (index)	WordNet id	Data type	Dimensions (H, W, C)	Memory
0	pomegranate (170)	n07768694	uint8	(64, 64, 3)	12,288 bytes
1	miniskirt (77)	n03770439	uint8	(64, 64, 3)	12,288 bytes
2	German shepherd (11)	n02106662	uint8	(64, 64, 3)	12,288 bytes


b'pomegranate' (class index: 170 - id: b'n07768694')
0
10
20
30
40
50
60
0
10
20
30
40
50
60


b'miniskirt, mini' (class index: 77 - id: b'n03770439')
0
10
20
30
40
50
60
0
10
20
30
40
50
60

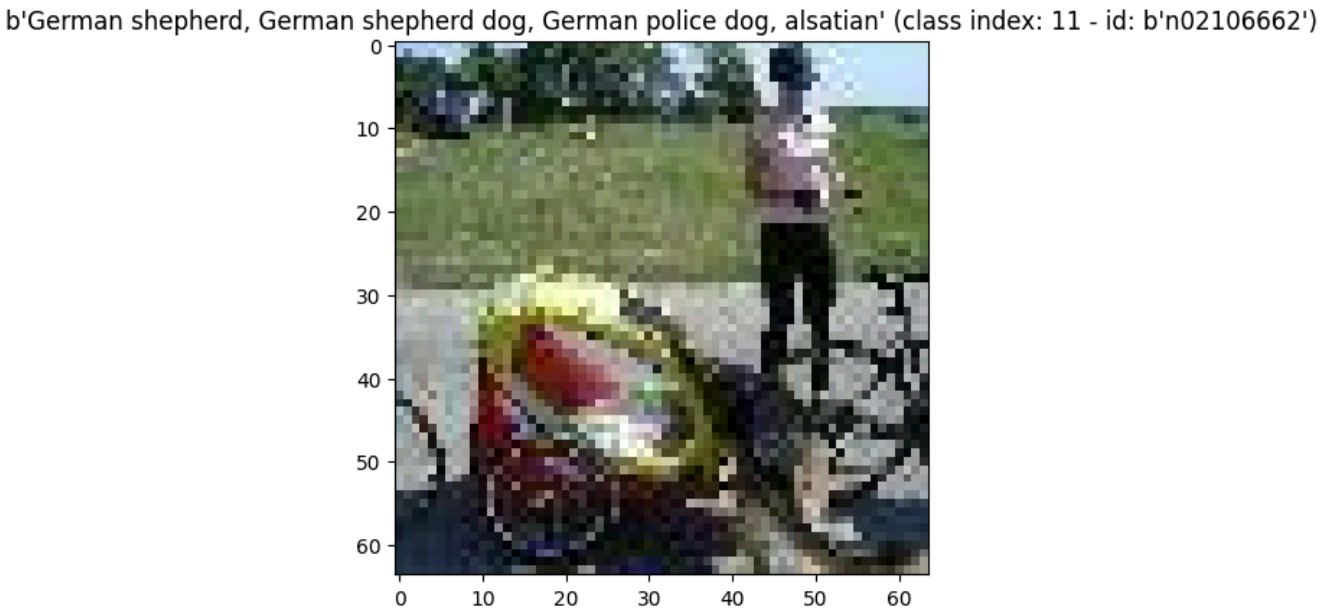


Figure 1. The three test images from the validation set, shown with their class labels: pomegranate (170), miniskirt (77) and German shepherd (11).

Each image was exported to `img_data/image_{0,1,2}.bin` as raw `uint8` bytes, flattened in (H, W, C) order to a 12,288-element array. The notebook checks that every file is exactly 12,288 bytes. `img_data/metadata.txt` records the index, dimensions and class of each image.

3. Model loading and inference (3.4)

The model is a 12-layer CNN with 770,216 parameters (2.94 MB as float32). It classifies one of the three test images correctly and reaches 24.30% Top-1 accuracy on the 10,000 validation images.

Model summary

Layer	Type	Output shape	Parameters
conv2d	Conv2D 5×5, 32 filters	(60, 60, 32)	2,432
conv2d_1	Conv2D 5×5, 32 filters	(56, 56, 32)	25,632
max_pooling2d	MaxPooling2D 2×2	(28, 28, 32)	0
conv2d_2	Conv2D 3×3, 64 filters	(26, 26, 64)	18,496
conv2d_3	Conv2D 3×3, 64 filters	(24, 24, 64)	36,928
max_pooling2d_1	MaxPooling2D 2×2	(12, 12, 64)	0
conv2d_4	Conv2D 3×3, 64 filters	(10, 10, 64)	36,928

Layer	Type	Output shape	Parameters
conv2d_5	Conv2D 3×3, 128 filters	(8, 8, 128)	73,856
max_pooling2d_2	MaxPooling2D 2×2	(4, 4, 128)	0
flatten	Flatten	(2048)	0
dense	Dense, ReLU	(256)	524,544
dense_1	Dense, softmax	(200)	51,400

The `dense` layer alone holds 68% of the parameters (524,544 of 770,216).

Inference on the three test images

Input pixels were scaled from `uint8` [0, 255] to `float32` [0, 1] before inference.

Image	Best guess (index)	Confidence	Actual class (index)	Correct
0	pomegranate (170)	0.7293	pomegranate (170)	Yes
1	cash machine (106)	0.1033	miniskirt (77)	No
2	rugby ball (128)	0.2106	German shepherd (11)	No

The model was not confident about the two images it got wrong (0.10 and 0.21). For image 2 we think the mistake is understandable: a person and a bicycle take up most of the picture and the dog is hard to see.

Validation accuracy

Metric	Accuracy (10,000 images)
Top-1	24.30%
Top-5	49.40%
Top-10	61.74%

How would we rate this model, and would we use it? Random guessing with 200 classes would give 0.5% Top-1, so 24.30% is about 49 times better than chance. The model has clearly learned something. But it is still wrong for three out of four images, and even if we allow ten guesses the correct class is missing 38% of the time. We would not use it for anything that needs one reliable answer. We think it is fine as a small and fast model for studying inference hardware, which is what the later labs use it for.

Classes and dataset size

The notebook prints all 200 class names. The training split has 100,000 images (500 per class) and the validation split has 10,000 images (50 per class), 110,000 in total.

Why is so much data important, and is our dataset small? The model has 770,216 parameters and only 100,000 training images, which is about 7.7 parameters per image. With so many parameters, a model with too little data can just memorize the training images instead of learning something general. We actually saw this in Section 6: the training accuracy kept going up after the validation accuracy stopped near 24%. Compared to current datasets ours is small. The original ImageNet classification set has about 1.2 million training images in 1,000 classes at a much higher resolution, while TinyImageNet has 500 images per class at 64 × 64.

Source for the ImageNet figures: [ILSVRC 2012 challenge page].

4. Model exploration (3.5)

As we go deeper into the model, the weights get bigger and the feature maps get smaller. The last convolution has 30 times more weight memory than the first one, but its output is 14 times smaller.

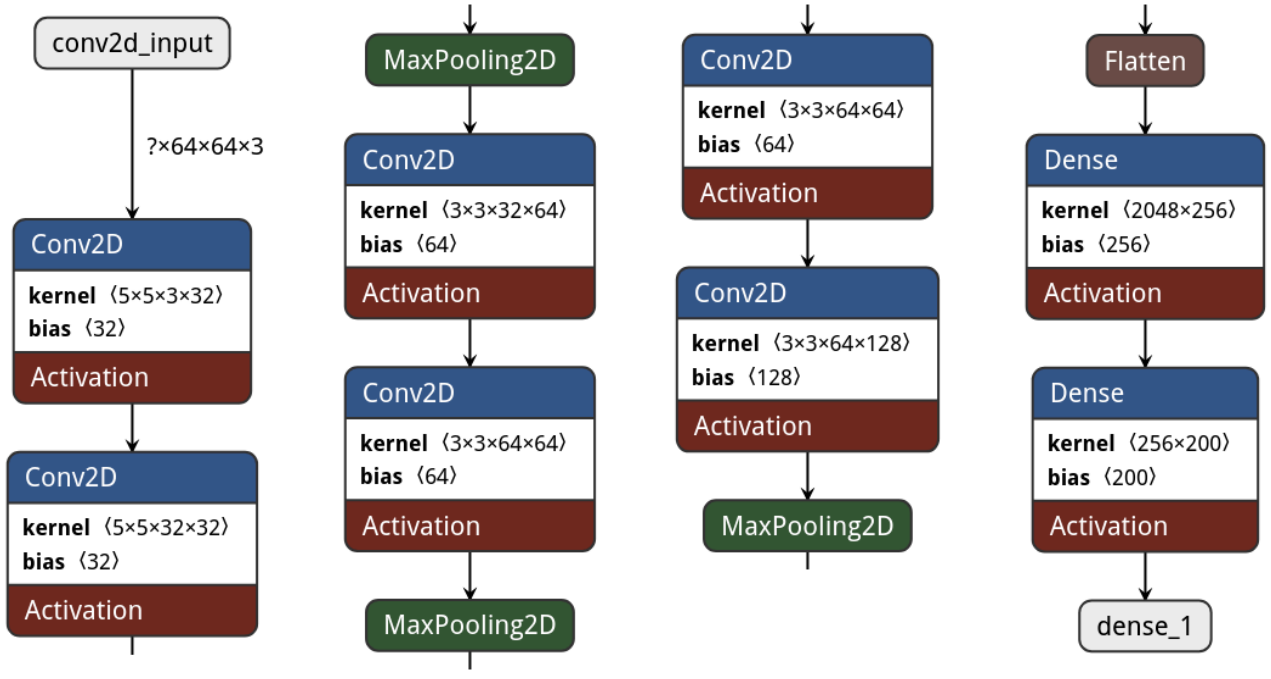


Figure 2. The model visualized in Netron: six convolutions, three max-pooling layers, a flatten layer and two dense layers. The Netron graph is one tall column; we split it into four columns that read from left to right so that it fits on one page.

Weights and feature maps of two layers

We compared `conv2d` (first layer, near the input) and `conv2d_5` (last convolution, near the output). All values are `float32`. Feature maps were computed for test image 0 (pomegranate).

	conv2d (near input)	conv2d_5 (near output)
Kernel dimensions (H, W, in, out)	(5, 5, 3, 32)	(3, 3, 64, 128)
Bias dimensions	(32,)	(128,)
Weight memory (kernel + bias)	9,600 + 128 = 9,728 bytes	294,912 + 512 = 295,424 bytes
Feature map dimensions (N, H, W, C)	(1, 60, 60, 32)	(1, 8, 8, 128)
Feature map memory	460,800 bytes	32,768 bytes

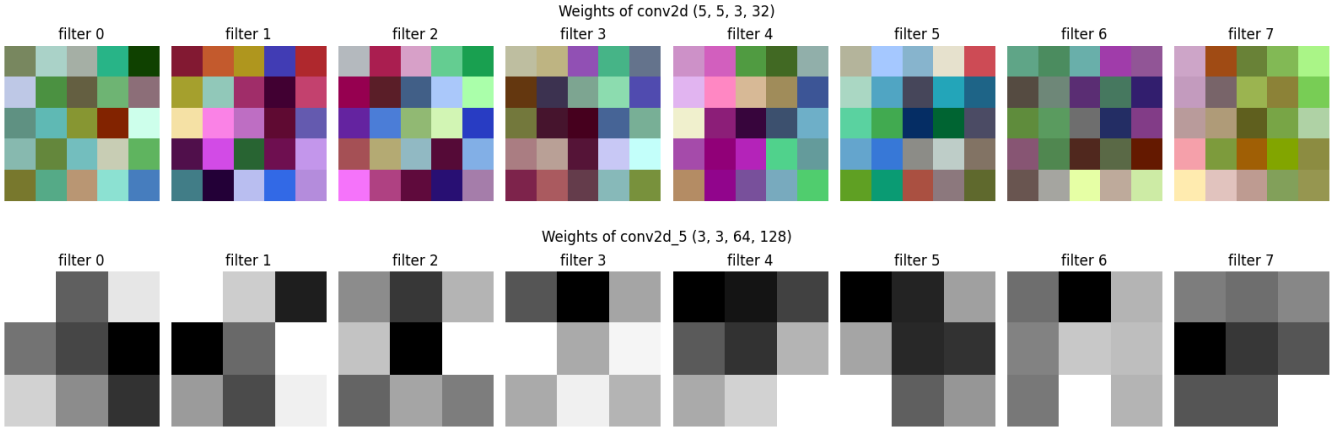


Figure 3. The first 8 filters of `conv2d` (top, shown as 5 × 5 RGB images) and of `conv2d_5` (bottom, the 3 × 3 slice for input channel 0).

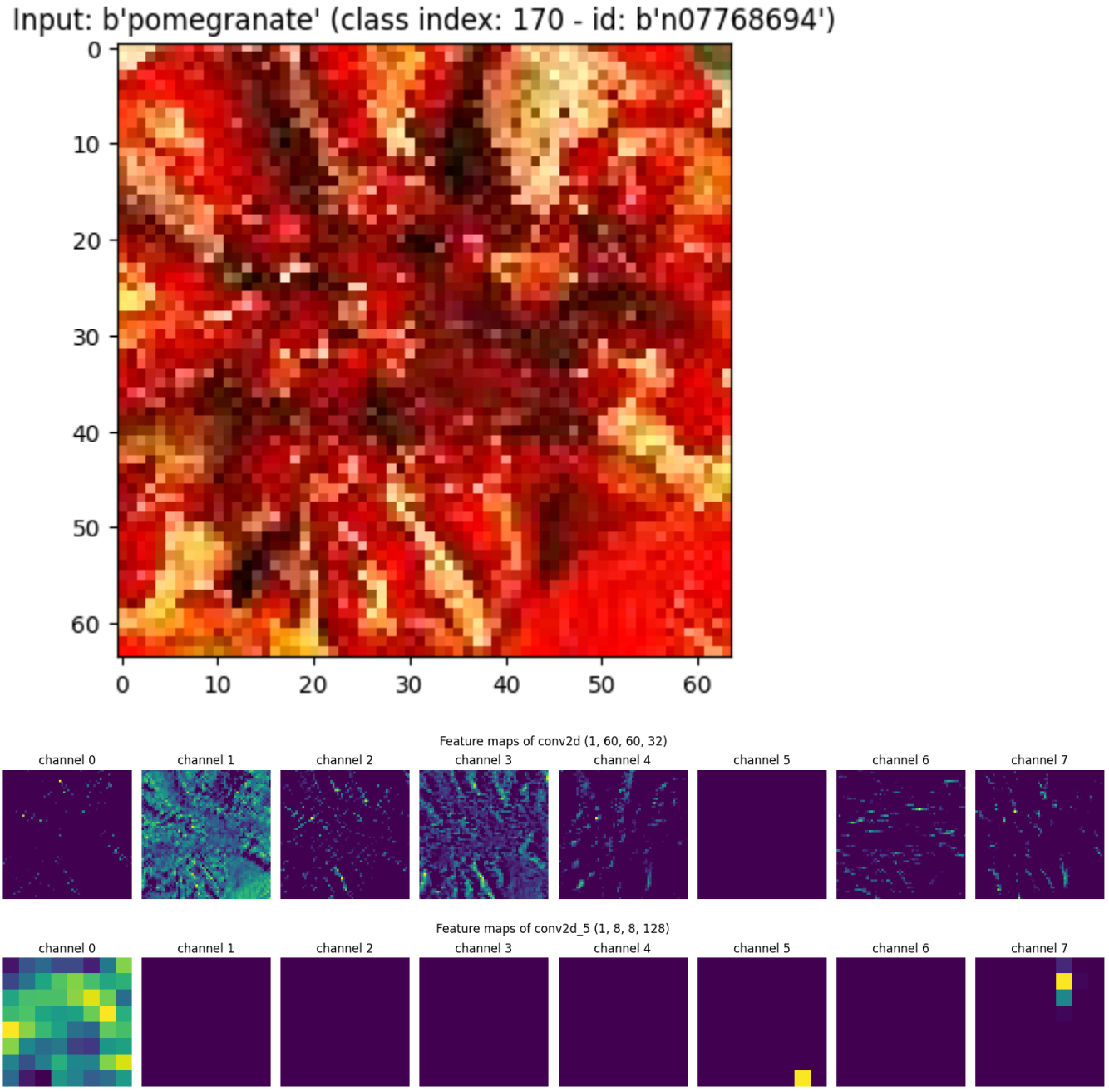


Figure 4. The input image (test image 0) and the first 8 feature map channels of `conv2d` (60 × 60) and `conv2d_5` (8 × 8).

For `conv2d` each filter is shown as a 5 × 5 RGB image. For `conv2d_5` each filter has 64 input channels, so the figure shows the 3 × 3 slice applied to input channel 0.

How do the feature maps change through the model? The height and width shrink from 60 × 60 to 8 × 8, while the number of channels grows from 32 to 128. In `conv2d` we can still recognize the texture and edges of the input image in several channels. In `conv2d_5` the 8 × 8 maps do not look like the image anymore, and five of the eight channels we plotted are completely zero. So the later feature maps are sparse and much more abstract.

How does this differ from the weights? The weights are fixed after training, so they are the same no matter what image we give the model. The feature maps change with the input: a channel that is all zero for this image could be active for a different one. The sizes also go in opposite directions as we go deeper, which the table above shows.

Is there a correlation with layer depth? Yes. The deeper layers have more filters and more input channels, so they have more weights, and they produce smaller maps with more channels. The filters in the first layer work directly on RGB pixels, so we can look at them as small color images. The deeper filters work on the outputs of earlier filters, so we could not interpret them just by looking at them.

Binary export

Files	Content	Format
`model_data/<layer>_weights.bin`, `<layer>_bias.bin`	Weights and biases of the 6 convolution and 2 dense layers	Raw float32, Keras layout: convolution (H, W, in, out), dense (in, out)
`img_data/test_input_{0,1,2}/<layer>_output.bin`	Output of all 12 layers for each test image	Raw float32, (1, H, W, C) or (1, N)

Two checks in the notebook confirm the export. The weight files total 3,080,864 bytes, which is 770,216 float32 values and equals the model's parameter count. The class predicted from each exported final-layer output matches the inference table in Section 3: class 170 (0.7293), class 106 (0.1033) and class 128 (0.2106).

5. Profiling with TensorBoard (3.6)

Convolution takes most of the model's computation time, and `conv2d_1` is the slowest single layer. However, for single-image inference we found that most of the 57–66 ms end-to-end latency is not the layers at all. It is the fixed cost of calling `model.predict`.

All measurements are on CPU. One warm-up inference was run before timing so that graph construction is not counted. The lab manual's `tensorflow_stats` view is named `framework_op_stats` in this TensorBoard version.

5.1 Single-image inference

Image	Profiled run (ms)	Median of 20 runs (ms)
0 pomegranate	66.32	64.63
1 miniskirt	61.01	57.52
2 German shepherd	100.32	57.29

Latency is the wall-clock time of one `model.predict` call: (end time − start time) × 1000. The three images take the same time within noise because the input size and the computation are identical. The 100.32 ms value is a one-off spike in a single measurement; its median is 57.29 ms.

5.2 Which layers take the time

In the profile of the three single inferences, the operations inside the model total about 8.8 ms (2.9 ms per image), split by type as follows.

Operation type	Total self-time (us)	Share of model operations
Conv2D (`_MklNativeFusedConv2D`)	6,717	76.2%
MaxPool	1,254	14.2%
MatMul (dense layers)	791	9.0%
Softmax	49	0.6%

Layer type that contributes most: convolution, with 76% of the model's computation time. We think the reason is the amount of arithmetic. For every output value a convolution has to do kernel height × kernel width × input channels multiply-accumulate operations (MACs), and the early layers have large output maps. That is far more work than pooling or dense layers. When we counted them, the six convolutions make up 131.0 million of the model's 131.6 million MACs.

The same profile also shows 14,151 us in `Dataset` operations. These belong to the input pipeline that `model.predict` builds on every call and are not model layers.

Layer that dominates: `conv2d_1`. With only three samples per operation the single-inference profile is noisy, so the per-layer values below come from the 1,000-image online profile (1,000 samples per operation) and from the batch-size-200 profile.

Layer	Online, per image (us)	Share	Batch 200, per image (us)	Share	MACs (millions)
conv2d	283	15.9%	45.0	7.7%	8.64
conv2d_1	538	30.3%	310.3	53.1%	80.28
max_pooling2d	93	5.2%	21.1	3.6%	0
conv2d_2	167	9.4%	62.8	10.7%	12.46
conv2d_3	187	10.5%	87.3	14.9%	21.23
max_pooling2d_1	59	3.3%	6.3	1.1%	0
conv2d_4	90	5.1%	17.3	3.0%	3.69
conv2d_5	94	5.3%	27.1	4.6%	4.72
max_pooling2d_2	46	2.6%	2.4	0.4%	0
dense	180	10.1%	3.7	0.6%	0.52
dense_1	40	2.3%	1.2	0.2%	0.05
Total	1,777		584.3		131.60

`conv2d_1` dominates because it applies 5 × 5 filters over 32 input channels (800 MACs per output value) to a 56 × 56 × 32 output, 80.28 million MACs or 61% of the whole model. `flatten` only reshapes and has no entry in the profile.

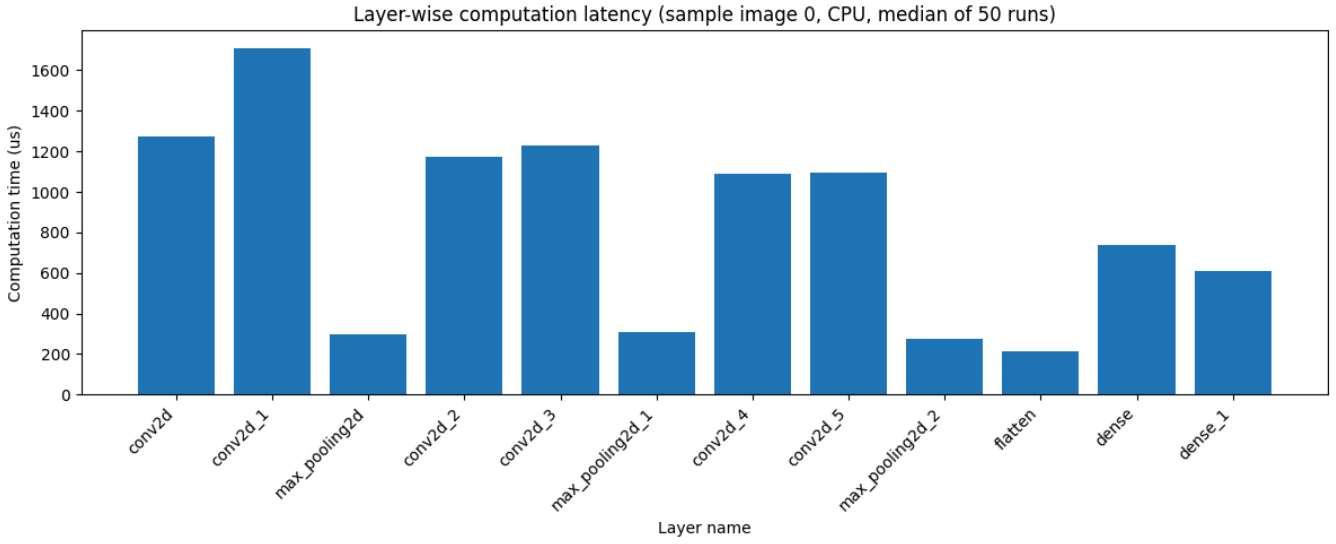


Figure 5. Layer-wise computation latency per image on the CPU, for online inference (left) and batch size 200 (right), from the TensorBoard profiler.

5.3 Online inference

Images	End-to-end latency (s)	Throughput (images/s)	Per image (ms)
10	0.603	16.58	60.30

Images	End-to-end latency (s)	Throughput (images/s)	Per image (ms)
100	6.201	16.13	62.01
1,000	56.329	17.75	56.33

Throughput = images ÷ latency. Latency grows linearly with the number of images, so the per-image cost stays near 56–62 ms.

5.4 Batch inference (1,000 images)

Batch size	End-to-end latency (s)	Throughput (images/s)	Per image (ms)	Speed-up over online
1 (online)	56.329	17.75	56.33	1.0×
20	4.464	224.01	4.46	12.6×
40	2.725	366.91	2.73	20.7×
100	1.681	594.88	1.68	33.5×
200	1.188	841.62	1.19	47.4×

The online and batch runs use the same 1,000 validation images, loaded into memory beforehand. `batch_size` is passed to `model.predict` explicitly; otherwise Keras splits the input into batches of 32.

Differences between batch sizes and possible reasons. The latency goes down as the batch gets bigger. From the profiles we found two reasons.

1. Fewer calls. Every `model.predict` call has a fixed cost. The time per call was 89, 109, 168 and 238 ms for batch sizes 20, 40, 100 and 200, which is roughly 70 ms per call plus 0.8 ms per image. Online inference pays this fixed cost 1,000 times, but batch 200 only pays it 5 times. In the 56.3 s online run, the model operations themselves only took 1.78 s (about 3%).
2. Less computation per image. The model operations for the same 1,000 images took 1.78 s online but only 0.58 s at batch 200. We think this is because the overhead of each operation is shared between all images in the batch, and the CPU can process many images in one vectorized call.

When to prefer each method. Online inference gives each result right after its input arrives. We would use it for interactive or real-time applications, where requests come one at a time and the delay of each request matters. Batch inference has a much higher

throughput, but a request has to wait until its batch is full and finished (238 ms for a batch of 200 in our test), and it needs more memory. We would use it for offline processing of a large dataset.

5.5 Kernels most critical for optimization

The convolution kernels are the ones to optimize, `conv2d_1` first. Convolution takes 76% of the model's computation time online and 94% at batch 200, and `conv2d_1` alone takes 30% and 53%.

Does the cost per layer make sense? For batch inference it does. Each convolution costs about 4–6 us per million MACs, so the time follows the amount of arithmetic, and `conv2d_1`, which has the most MACs, is the slowest. For online inference the costs are less proportional. For example, `dense` has only 1/150 of the MACs of `conv2d_1` but takes one third of its time. Our explanation is that with one image every operation has a fixed overhead, and for a small layer this overhead is larger than the actual work. In a batch the overhead mostly disappears, and `dense` drops from 180 us to 3.7 us per image. The pooling layers have no multiplications, but they still have to read every input value, so their cost is small but not zero.

6. Training (3.7)

When we trained the same architecture from scratch, it reached 24–25% validation Top-1 after about 8 epochs. This is the same level as the provided model (24.30%), and training for more epochs did not improve it.

Setup: Adam optimizer with learning rate 3e-4, categorical cross-entropy loss, the same random seed for every run, and the template's `EarlyStopping` callback (monitor `val_loss`, `min_delta` 0.01, patience 2). Every run starts from a freshly initialized copy of the model. We lowered the learning rate from the Adam default of 1e-3 because in our earlier attempt some runs stopped learning, with the loss fixed at ln(200) = 5.298. None of the six runs below showed that behaviour.

6.1 Batch size (up to 20 epochs)

Batch size	Epochs run	Best epoch	Best validation Top-1	Best validation Top-5	Final training Top-1	Time per epoch (s)
32	8	6	24.40%	48.92%	35.25%	15.9
64	8	8	24.99%	49.28%	32.90%	11.8
128	12	10	24.82%	48.92%	37.31%	9.1

All three runs were stopped by early stopping before they reached 20 epochs. The accuracies differ by at most 0.6 percentage points, and we only ran each setting once, so we do not think we can say that one batch size is more accurate than another. What we can see is the speed: larger batches make each epoch faster (15.9 s down to 9.1 s), but they update the weights fewer times per epoch (3,125 steps at batch 32 and 782 at batch 128). This is probably why batch 128 needed more epochs.

We used the template's batch sizes of 32, 64 and 128 instead of the manual's examples of 2 to 16, which would need up to 50,000 steps per epoch.

6.2 Number of epochs (batch size 32)

Epochs requested	Epochs run	Best epoch	Best validation Top-1	Best validation Top-5	Final model Top-1	Final model Top-5	Final training Top-1
3	3	3	18.93%	42.96%	18.93%	42.96%	17.48%
10	10	9	24.41%	48.98%	24.00%	47.83%	39.48%
100	8	8	23.86%	49.52%	23.86%	49.52%	34.58%

a. How does the number of epochs affect validation accuracy? The accuracy goes up quickly at the beginning and then flattens out. After 3 epochs the model is still underfitted (18.93% Top-1, and the validation loss was still falling, from 4.50 to 3.68). After 8 to 10 epochs it reaches about 24% and does not get better.

b. Minimum number of epochs for the best validation accuracy. In our six runs the best validation Top-1 always came between epoch 6 and epoch 10: epochs 6, 8 and 9 at batch 32, epoch 8 at batch 64 and epoch 10 at batch 128. So for batch size 32, about 8 epochs are enough.

c. What happens with a large number of epochs? The run where we asked for 100 epochs stopped after 8. This is because of the template's `EarlyStopping` callback, which ends training when the validation loss has not improved by at least 0.01 for two epochs in a row (in our run it was 3.4491, 3.4884, 3.4454 in epochs 6 to 8). The reason behind it is overfitting. In the 10-epoch run the training accuracy keeps going up (34.0% to 39.5% over epochs 8 to 10), but the validation loss starts to rise again (3.44, 3.51, 3.64) and the validation Top-1 stays near 24%. The model starts to memorize the 100,000 training images instead of learning features that work on new images, so more epochs only make the gap bigger. One limitation is that early stopping was on, so none of our runs went past epoch 12. What happens at very large epoch counts is our conclusion from this trend, not something we measured.

6.3 CPU versus GPU

Device	Time per epoch (batch 32)	Per step
CPU (`linuxvdi-f26-19`)	318 s	102 ms
GPU (NVIDIA A40-8Q)	about 15 s	5 ms

The GPU is about 20 times faster. On the CPU we only ran the first epoch (it reached 8.19% validation Top-1) and then moved to the GPU VM. Our experiments asked for 173 epochs in total, which would have taken more than 15 hours on the CPU without early stopping.

Appendix: figures and TensorBoard screenshots to insert

Figures 1 to 5 are in the report. The TensorBoard screenshots are in this appendix as Figures A1 to A6 and Table A7. Notebook figures can be copied from the cell outputs; TensorBoard screenshots come from the Profile tab, tool `framework_op_stats`.

Screenshot	TensorBoard run	Supports
Pie charts of self-time by type and by operation	`2026_10_07_16_07_55` (single inference, 3 images)	Section 5.2
Framework Operations table, ranks 1–11	`2026_10_07_16_07_55`	Section 5.2
Host-side operations grouped by type	`2026_10_07_16_07_55`	Section 5.2, type table
Framework Operations table, ranks 1–14	`2026_10_07_16_14_51` (online, 1,000 images)	Section 5.2, online column
Host-side operations grouped by type	`2026_10_07_16_14_51`	Section 5.4, reason 1 and 2
Framework Operations table, ranks 1–11	`2026_10_07_16_16_16` (batch size 200)	Section 5.2, batch column
Host-side operations grouped by type	`2026_10_07_16_16_16`	Section 5.4 and 5.5

Single inference (run `2026_10_07_16_07_55`, 3 images)

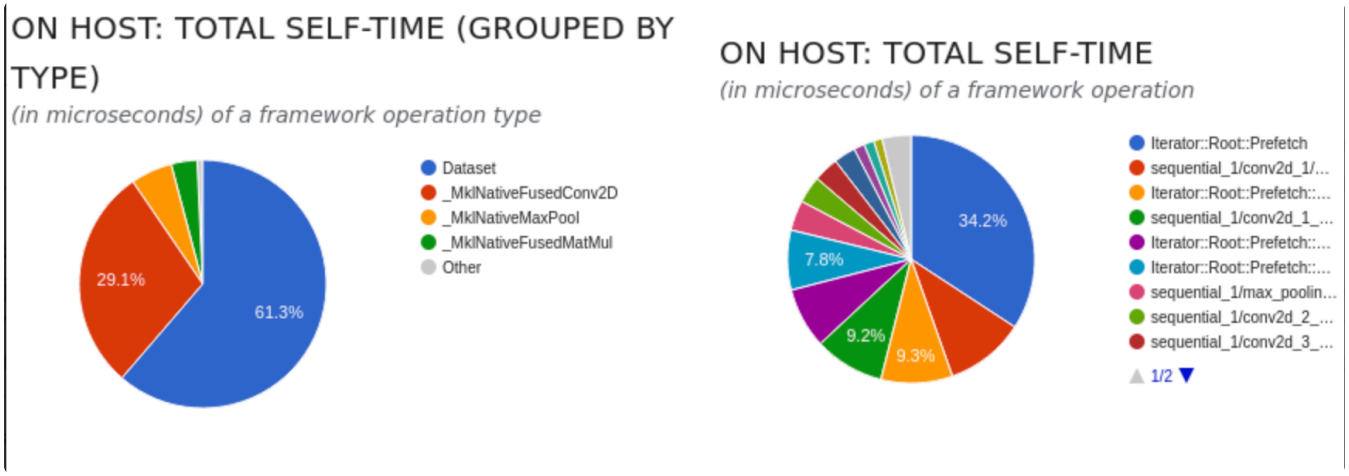


Figure A1. Total self-time on the host by operation type (left) and by operation (right) for the three single inferences.

Rank	Host/ device	Type	Operation	#Occurrences	Total time (us)	Avg. time (us)	Total self-time (us)	Avg. self-time (us)	Total self-time on Device (%)	Cumulative total-self time on Device (%)	Total self-time on Host (%)	Cumulative total-self time on Host (%)
1	Host	Dataset	Iterator::Root::Prefetch	3	7,889	2,630	7,889	2,630	0%	0%	34.2%	34.2%
2	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1/Relu	3	2,399	800	2,399	800	0%	0%	10.4%	44.6%
3	Host	Dataset	Iterator::Root::Prefetch::ParallelMapV2	2	2,156	1,078	2,156	1,078	0%	0%	9.3%	53.9%
4	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1_2/Relu	3	2,117	706	2,117	706	0%	0%	9.2%	63.1%
5	Host	Dataset	Iterator::Root::Prefetch::ParallelMapV2::Zip[0]::FlatMap::Prefetch	2	1,820	910	1,820	910	0%	0%	7.9%	71%
6	Host	Dataset	Iterator::Root::Prefetch::ParallelMapV2::Zip[0]::FlatMap::Prefetch::Map	4	1,809	452	1,798	450	0%	0%	7.8%	78.8%
7	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_1/MaxPool2d	3	907	302	907	302	0%	0%	3.9%	82.7%
8	Host	_MklNativeFusedConv2D	sequential_1/conv2d_2_1/Relu	3	817	272	817	272	0%	0%	3.5%	86.2%
9	Host	_MklNativeFusedConv2D	sequential_1/conv2d_3_1/Relu	3	755	252	755	252	0%	0%	3.3%	89.5%
10	Host	_MklNativeFusedMatMul	sequential_1/dense_1/Relu	3	658	219	658	219	0%	0%	2.9%	92.4%
11	Host	_MklNativeFusedConv2D	sequential_1/conv2d_5_1/Relu	3	325	108	325	108	0%	0%	1.4%	93.8%

Figure A2. Framework operations for the single inferences, sorted by total self-time (ranks 1 to 11).

Host-side TensorFlow operations (grouped by TYPE)

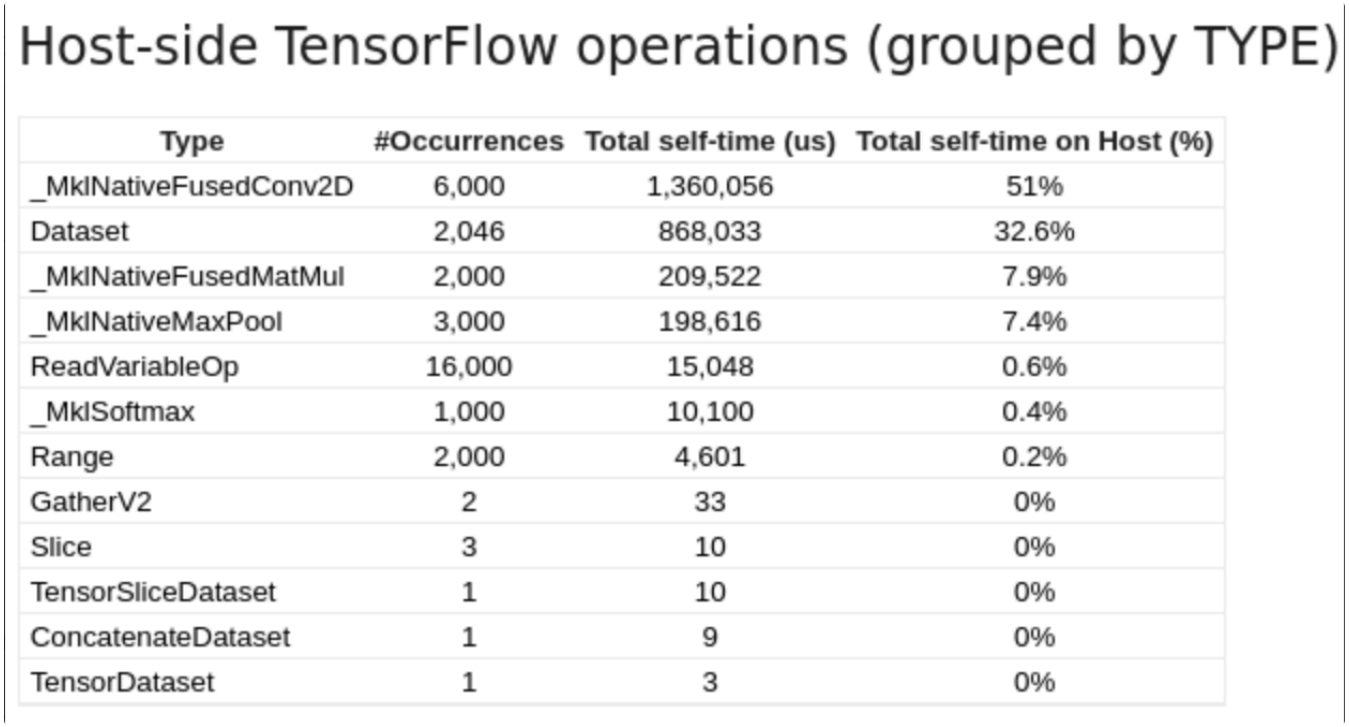
Type	#Occurrences	Total self-time (us)	Total self-time on Host (%)
Dataset	36	14,151	61.3%
_MklNativeFusedConv2D	18	6,717	29.1%
_MklNativeMaxPool	9	1,254	5.4%
_MklNativeFusedMatMul	6	791	3.4%
_MklSoftmax	3	49	0.2%
ReadVariableOp	48	42	0.2%
ConcatenateDataset	1	14	0.1%
Range	6	14	0.1%
GatherV2	1	12	0.1%
TensorDataset	1	11	0%
TensorSliceDataset	1	11	0%
Slice	3	10	0%

Figure A3. Host-side operations grouped by type for the single inferences. This is the source of the type table in Section 5.2.

Online inference (run `2026_10_07_16_14_51`, 1,000 images)

Rank	Host/ device	Type	Operation	#Occurrences	Total time (us)	Avg. time (us)	Total self-time (us)	Avg. self-time (us)	Total self-time on Device (%)	Cumulative total-self time on Device (%)	Total self-time on Host (%)	Cumulative total-self time on Host (%)
1	Host	Dataset	Iterator::Root::Prefetch	1,000	781,436	781	781,436	781	0%	0%	29.3%	29.3%
2	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1_2/Relu	1,000	538,357	538	538,357	538	0%	0%	20.2%	49.5%
3	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1/Relu	1,000	283,190	283	283,190	283	0%	0%	10.6%	60.1%
4	Host	_MklNativeFusedConv2D	sequential_1/conv2d_3_1/Relu	1,000	187,213	187	187,213	187	0%	0%	7%	67.1%
5	Host	_MklNativeFusedMatMul	sequential_1/dense_1/Relu	1,000	180,011	180	180,011	180	0%	0%	6.8%	73.9%
6	Host	_MklNativeFusedConv2D	sequential_1/conv2d_2_1/Relu	1,000	167,079	167	167,079	167	0%	0%	6.3%	80.2%
7	Host	_MklNativeFusedConv2D	sequential_1/conv2d_5_1/Relu	1,000	93,799	94	93,799	94	0%	0%	3.5%	83.7%
8	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_1/MaxPool2d	1,000	93,157	93	93,157	93	0%	0%	3.5%	87.2%
9	Host	_MklNativeFusedConv2D	sequential_1/conv2d_4_1/Relu	1,000	90,418	90	90,418	90	0%	0%	3.4%	90.6%
10	Host	Dataset	Iterator::Root	1,000	866,773	867	85,336	85	0%	0%	3.2%	93.8%
11	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_1_2/MaxPool2d	1,000	59,095	59	59,095	59	0%	0%	2.2%	96%
12	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_2_1/MaxPool2d	1,000	46,364	46	46,364	46	0%	0%	1.7%	97.7%
13	Host	_MklNativeFusedMatMul	sequential_1/dense_1_2/BiasAdd	1,000	29,511	30	29,511	30	0%	0%	1.1%	98.8%
14	Host	_MklSoftmax	sequential_1/dense_1_2/Softmax	1,000	10,100	10	10,100	10	0%	0%	0.4%	99.2%

Figure A4. Framework operations for 1,000 online inferences (ranks 1 to 14). The average self-time column is the source of the online values in Section 5.2.

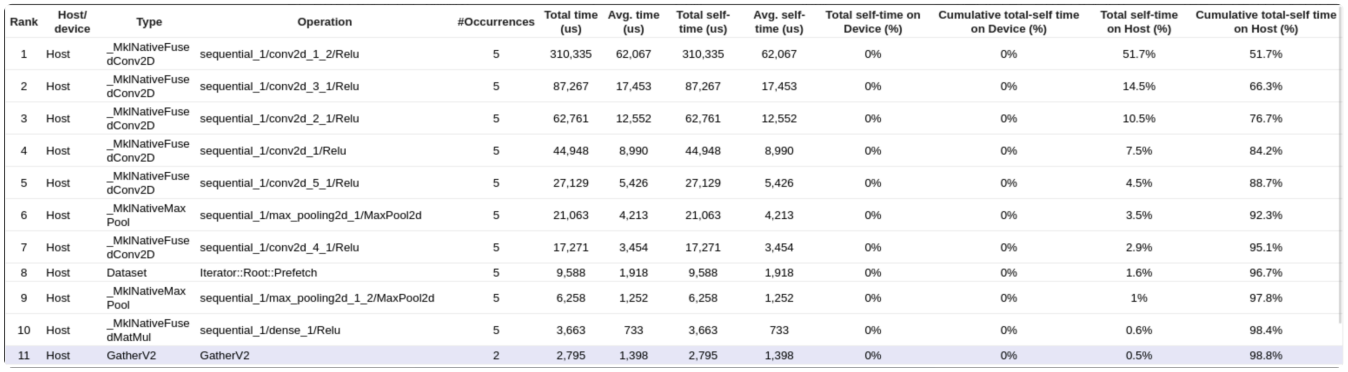


Host-side TensorFlow operations (grouped by TYPE)

Type	#Occurrences	Total self-time (us)	Total self-time on Host (%)
_MklNativeFusedConv2D	6,000	1,360,056	51%
Dataset	2,046	868,033	32.6%
_MklNativeFusedMatMul	2,000	209,522	7.9%
_MklNativeMaxPool	3,000	198,616	7.4%
ReadVariableOp	16,000	15,048	0.6%
_MklSoftmax	1,000	10,100	0.4%
Range	2,000	4,601	0.2%
GatherV2	2	33	0%
Slice	3	10	0%
TensorSliceDataset	1	10	0%
ConcatenateDataset	1	9	0%
TensorDataset	1	3	0%

Figure A5. Host-side operations grouped by type for 1,000 online inferences.

Batch inference (run `2026_10_07_16_16_16`, batch size 200)



Rank	Host/ device	Type	Operation	#Occurrences	Total time (us)	Avg. time (us)	Total self-time (us)	Avg. self-time (us)	Total self-time on Device (%)	Cumulative total-self time on Device (%)	Total self-time on Host (%)	Cumulative total-self time on Host (%)
1	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1_2/Relu	5	310,335	62,067	310,335	62,067	0%	0%	51.7%	51.7%
2	Host	_MklNativeFusedConv2D	sequential_1/conv2d_3_1/Relu	5	87,267	17,453	87,267	17,453	0%	0%	14.5%	66.3%
3	Host	_MklNativeFusedConv2D	sequential_1/conv2d_2_1/Relu	5	62,761	12,552	62,761	12,552	0%	0%	10.5%	76.7%
4	Host	_MklNativeFusedConv2D	sequential_1/conv2d_1/Relu	5	44,948	8,990	44,948	8,990	0%	0%	7.5%	84.2%
5	Host	_MklNativeFusedConv2D	sequential_1/conv2d_5_1/Relu	5	27,129	5,426	27,129	5,426	0%	0%	4.5%	88.7%
6	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_1/MaxPool2d	5	21,063	4,213	21,063	4,213	0%	0%	3.5%	92.3%
7	Host	_MklNativeFusedConv2D	sequential_1/conv2d_4_1/Relu	5	17,271	3,454	17,271	3,454	0%	0%	2.9%	95.1%
8	Host	Dataset	Iterator::Root::Prefetch	5	9,588	1,918	9,588	1,918	0%	0%	1.6%	96.7%
9	Host	_MklNativeMaxPool	sequential_1/max_pooling2d_1_2/MaxPool2d	5	6,258	1,252	6,258	1,252	0%	0%	1%	97.8%
10	Host	_MklNativeFusedMatMul	sequential_1/dense_1/Relu	5	3,663	733	3,663	733	0%	0%	0.6%	98.4%
11	Host	GatherV2	GatherV2	2	2,795	1,398	2,795	1,398	0%	0%	0.5%	98.8%

Figure A6. Framework operations for batch size 200 (5 calls, ranks 1 to 11). The average self-time divided by 200 is the source of the batch values in Section 5.2.

Table A7. Host-side operations grouped by type for batch size 200, copied from the TensorBoard profile.

Type	Count	Total self-time (us)	Host share
_MklNativeFusedConv2D	30	549,710	91.6%
_MklNativeMaxPool	15	29,744	5%
Dataset	54	12,819	2.1%
_MklNativeFusedMatMul	10	4,561	0.8%
GatherV2	2	2,795	0.5%
_MklSoftmax	5	295	0%
Range	10	41	0%
TensorSliceDataset	1	11	0%
Slice	1	4	0%

Operation names in the profile carry an extra suffix compared with `model.summary()`: layer `conv2d` appears as `sequential_1/conv2d_1/Relu`, layer `conv2d_1` as `sequential_1/conv2d_1_2/Relu`, layer `dense` as `sequential_1/dense_1/Relu`, and layer `dense_1` as `sequential_1/dense_1_2/BiasAdd` plus `Softmax`. Each convolution is fused with its bias add and ReLU into one `_MklNativeFusedConv2D` operation.

In the batch-200 profile the rows for `max_pooling2d_2` and `dense_1` were below the captured part of the table. Their values in Section 5.2 are the per-type totals minus the visible rows.